

Universidad Autónoma de Nuevo León
Facultad de Ciencias Físico Matemáticas

# Laboratorio de Programación Orientada a Objetos

Avance — Laboratorio 10: Concurrencia con Hilos (Threads)

**Alumno:** Daniel Antonio Chávez Peña
**Matrícula:** 1956765
**Materia:** Laboratorio de Programación Orientada a Objetos

## Descripción de la práctica

Esta práctica tiene como objetivo introducir la programación concurrente en Java mediante la creación de hilos de dos formas distintas: extendiendo la clase `Thread` e implementando la interfaz `Runnable`. También se estudió el ciclo de vida de un hilo (NEW, RUNNABLE, RUNNING, BLOCKED/WAITING y TERMINATED) y el uso de `synchronized` para proteger datos que comparten varios hilos.

El escenario es la continuación del sistema RPG de laboratorios anteriores, donde las acciones se ejecutaban una por una, en orden. Para simular un torneo real, en el que varios combates ocurren al mismo tiempo en distintos campos de batalla, se creó la clase CombateConcurrente, que implementa Runnable y ejecuta en su método run() las rondas de ataque y defensa entre dos personajes hasta que uno queda derrotado, usando Thread.sleep(500) para simular el tiempo que tarda cada ronda. Cada combate registra su resultado en TablaResultados, un recurso compartido por todos los hilos cuyo método registrar() es synchronized, de modo que solo un hilo a la vez modifica la lista.

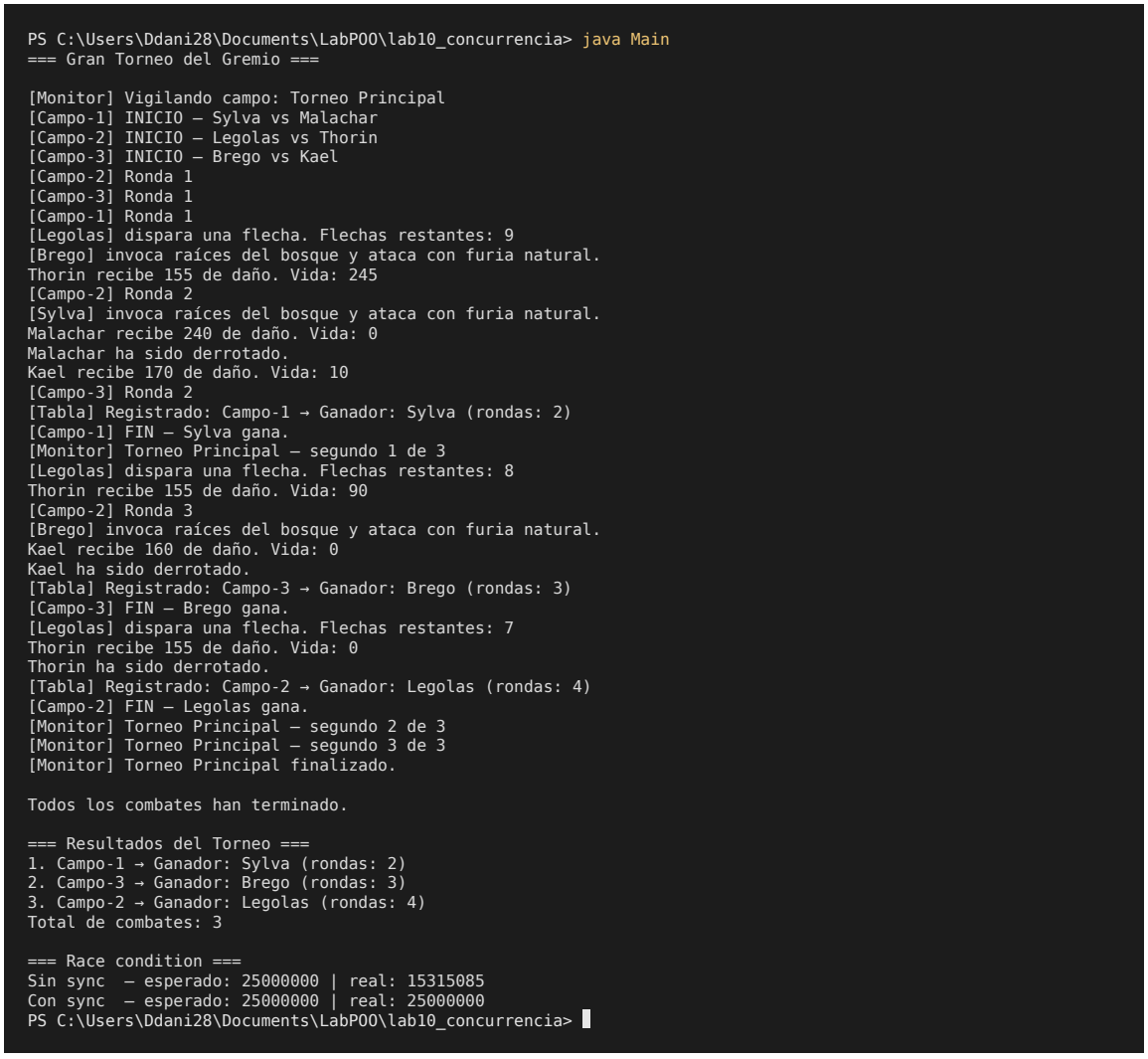
Además se implementó MonitorTorneo, que extiende directamente Thread y reporta cada segundo el estado del campo del torneo, y ContadorCompartido, que permite comparar un método sin protección (incrementar) contra uno sincronizado (incrementarSeguro) para observar el problema de la race condition. Todos los hilos manejan InterruptedException restaurando la bandera de interrupción con Thread.currentThread().interrupt().

En la clase Main se demostraron cuatro bloques: un torneo con tres combates simultáneos iniciados con start(), un monitor del torneo como hilo independiente, la espera de todos los hilos con join() antes de mostrar la tabla de resultados y la demostración de la race condition con cinco hilos. En esta última se aumentó el número de incrementos por hilo a 5,000,000, porque con solo 1,000 cada hilo termina antes de que arranque el siguiente y la pérdida de actualizaciones casi no se observa.

Finalmente, se completó la jerarquía de excepciones del Laboratorio 07 (RpgException como base chequeada, de la que heredan PersonajeDerrotadoException, RecursoInsuficienteException y AccionInvalidaException) y se corrigieron Arquero y Guerrero para que lancen las excepciones con throw en lugar de solo registrarlas, de forma que CombateConcurrente pueda capturarlas con catch (RpgException) y terminar el combate correctamente.

## Evidencia de ejecución

A continuación se muestra la salida en consola obtenida al ejecutar la clase principal Main, donde se observan los tres combates corriendo de forma simultánea (con mensajes intercalados), el monitor del torneo, la tabla de resultados y la demostración de la race condition. El orden de los mensajes y el valor del contador sin synchronized cambian en cada ejecución.



## Conclusión

Esta práctica me ayudó a entender cómo se ejecutan varias tareas al mismo tiempo en Java. Con los tres combates corriendo en hilos separados, los mensajes salen mezclados y el orden cambia en cada ejecución, pero la tabla de resultados siempre queda completa porque registrar() está sincronizado. También quedó claro para qué sirve join(): sin esperar a que terminen los hilos, el programa principal podría mostrar la tabla antes de que los combates acaben.

Además, comprendí la diferencia entre extender Thread e implementar Runnable: Runnable es más flexible porque la clase puede heredar de otra y la tarea queda separada del hilo que la ejecuta. Por último, la demostración del contador compartido mostró una race condition real: sin synchronized el resultado fue de 15,315,085 en lugar de 25,000,000 porque los hilos pisaban las actualizaciones de los demás, y con synchronized el resultado fue siempre el correcto.